

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Social Media Feed Processing

A social media platform processes posts using a recursive algorithm $T(n) = T(n-1) + n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive the final complexity. Analyze how performance degrades with large inputs. Interpret the result using asymptotic notation.

Q2. Smart Warehouse Inventory System

A warehouse system processes inventory using the recurrence $T(n) = 4T(n/2) + n \log n$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and analyze the space complexity.

Code of Conduct

I __Nithyasri.S_ 192421347 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Social Media Feed Processing

Problem Statement

A social media platform processes user posts using the recursive algorithm:

$$T(n) = T(n-1) + n$$

Apply the **Substitution Method** to solve the recurrence relation. Expand it step-by-step and derive the final time complexity. Analyze how the performance degrades for large inputs. Interpret the result using asymptotic notation.

Answer

Introduction

Social media applications such as Facebook, Instagram, Twitter (X), and LinkedIn continuously process millions of user posts. Suppose the system processes one new post at each recursive call while performing an additional amount of work proportional to the number of posts currently present.

The recurrence relation is:

$$T(n) = T(n-1) + n$$

where

- **T(n)** = Time required to process **n** posts.
- **T(n-1)** = Time required to process the previous **n-1** posts.
- **n** = Additional processing work performed at the current step.

The objective is to determine the overall running time using the **Substitution Method**.

Substitution Method

The substitution method solves a recurrence by repeatedly expanding it until a base case is reached.

Given,

$$T(n) = T(n-1) + n$$

Step 1: Expand Once

$$T(n) = T(n-1) + n \quad T(n) = T(n-1) + n \quad T(n) = T(n-1) + n$$

Replace

$$T(n-1) = T(n-2) + (n-1) \quad T(n-1) = T(n-2) + (n-1) \quad T(n-1) = T(n-2) + (n-1)$$

Therefore,

$$T(n) = T(n-2) + (n-1) + n \quad T(n) = T(n-2) + (n-1) + n \quad T(n) = T(n-2) + (n-1) + n$$

Step 2: Expand Again

Replace

$$T(n-2) = T(n-3) + (n-2) \quad T(n-2) = T(n-3) + (n-2) \quad T(n-2) = T(n-3) + (n-2)$$

Substituting,

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad T(n) = T(n-3) + (n-2) + (n-1) + n \quad T(n) = T(n-3) + (n-2) + (n-1) + n$$

Step 3: Continue Expansion

Again,

$$T(n-3) = T(n-4) + (n-3) \quad T(n-3) = T(n-4) + (n-3) \quad T(n-3) = T(n-4) + (n-3)$$

Hence,

$$T(n) = T(n-4) + (n-3) + (n-2) + (n-1) + n \quad T(n) = T(n-4) + (n-3) + (n-2) + (n-1) + n \quad T(n) = T(n-4) + (n-3) + (n-2) + (n-1) + n$$

General Expansion

After k expansions,

$$T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + n \quad T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + n \quad T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + n$$

Continue until

$$n-k=1 \quad n-k=1 \quad n-k=1$$

Therefore,

$$k=n-1 \quad k=n-1 \quad k=n-1$$

Finally,

$$T(n) = T(1) + 2 + 3 + 4 + \dots + n \quad T(n) = T(1) + 2 + 3 + 4 + \dots + n \quad T(n) = T(1) + 2 + 3 + 4 + \dots + n$$

Assume

$$T(1) = 1 \quad T(1) = 1 \quad T(1) = 1$$

Hence,

$$T(n) = 1 + 2 + 3 + \dots + n \quad T(n) = 1 + 2 + 3 + \dots + n \quad T(n) = 1 + 2 + 3 + \dots + n$$

Sum of First n Natural Numbers

We know,

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \quad 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \quad 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Therefore,

$$T(n) = \frac{n(n+1)}{2} \quad T(n) = \frac{n(n+1)}{2} \quad T(n) = \frac{n(n+1)}{2}$$

Expanding,

$$T(n) = \frac{n^2 + n}{2} \quad T(n) = \frac{n^2 + n}{2} \quad T(n) = \frac{n^2 + n}{2}$$

Ignoring constants and lower-order terms,

$$T(n) = \Theta(n^2) \quad T(n) = \Theta(n^2) \quad T(n) = \Theta(n^2)$$

Verification Using Small Values

$$n = 1$$

$$T(1) = 1 \quad T(1) = 1 \quad T(1) = 1$$

$$n = 2$$

$$T(2) = T(1) + 2 \quad T(2) = T(1) + 2 \quad T(2) = T(1) + 2 = 1 + 2 = 3 = 1 + 2 = 3 = 1 + 2 = 3$$

$$n = 3$$

$$T(3) = T(2) + 3 \quad T(3) = T(2) + 3 \quad T(3) = T(2) + 3 = 3 + 3 = 6 = 3 + 3 = 6 = 3 + 3 = 6$$

n = 4

$$T(4)=6+4=10 \quad T(4)=6+4=10 \quad T(4)=6+4=10$$

n = 5

$$T(5)=10+5=15 \quad T(5)=10+5=15 \quad T(5)=10+5=15$$

Sequence obtained:

1, 3, 6, 10, 15, 3, 6, 10, 15, 3, 6, 10, 15

These values match the triangular number formula

$$\frac{n(n+1)}{2}$$

Performance Analysis

For every new post, the algorithm performs additional work proportional to the current number of posts.

Example:

Number of Posts	Approximate Operations
10	55
100	5,050
1,000	500,500
10,000	50,005,000

The number of operations increases rapidly as the number of posts grows.

Thus, the algorithm becomes inefficient for large-scale social media platforms.

Asymptotic Analysis

Best Case

$$\Omega(n^2)$$

Average Case

$\Theta(n^2)$

Worst Case

$O(n^2)$

Performance Degradation

As the number of posts increases,

- Recursive calls increase.
- Additional processing work increases linearly.
- Total execution time increases quadratically.
- Memory usage also increases because recursive calls remain on the call stack.

Hence, the algorithm does not scale efficiently for millions of users.

Space Complexity

Each recursive call occupies stack memory.

Maximum recursion depth:

n

Therefore,

Space Complexity = $O(n)$

Final Result

Time Complexity

$\Theta(n^2)$

Space Complexity

$O(n)$

Conclusion

Using the substitution method, the recurrence

$$T(n) = T(n-1) + n$$

expands into the sum of natural numbers. The final solution is

$$T(n) = \frac{n(n+1)}{2}$$

Therefore,

$$T(n) = \Theta(n^2)$$

The algorithm becomes slower as the number of posts increases because the running time grows quadratically.

Q2. Smart Warehouse Inventory System

Problem Statement

A smart warehouse inventory system processes inventory using the recurrence

$$T(n) = 4T\left(\frac{n}{2}\right) + n \log n$$

Apply the **Master Theorem** to determine the time complexity. Clearly identify the parameters and analyze the space complexity.

Answer

Introduction

Modern warehouses process inventory using recursive divide-and-conquer algorithms.

The warehouse database is divided into smaller sections.

Each section is processed independently before combining the results.

The recurrence relation is

$$T(n) = 4T\left(\frac{n}{2}\right) + n \log n$$

where

- $T(n)$ = Time to process n inventory records.
- $4T(n/2)$ = Four recursive calls.
- $n \log n$ = Time required to merge/process results.

Applying Master Theorem

The standard form is

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Compare with

$$T(n) = 4T(n/2) + n \log n$$

Therefore,

$$a=4, b=2, f(n)=n \log n$$

Step 1: Compute

$$n \log_b a^{\log_b a} n \log_b a$$

Substitute values

$$= n \log 24 = n^{\log_2 4} = n \log 24$$

Since

$$\log 24 = 2 \log_2 4 = 2 \log 24 = 2$$

Therefore,

$$n \log_b a = n 2 n^{\log_b a} = n^2 n \log_b a = n^2$$

Step 2: Compare

Compare

$$f(n) = n \log n f(n) = n \log n f(n) = n \log n$$

with

$$n^2 n^2$$

Since

$$n \log n = O(n^{2-\epsilon}) n \log n = O(n^{2-\epsilon})$$

for

$$\epsilon = 1 \text{ } \epsilon = 1$$

the function $f(n)f(n)f(n)$ grows polynomially slower than $n^2 n^2$.

Hence,

Master Theorem **Case 1** applies.

Master Theorem Case 1

If

$$f(n) = O(n \log b a - \epsilon) \quad f(n) = O(n^{\{\log_b a - \epsilon\}}) \quad f(n) = O(n \log b a - \epsilon)$$

then

$$T(n) = \Theta(n \log b a) \quad T(n) = \Theta(n^{\{\log_b a\}}) \quad T(n) = \Theta(n \log b a)$$

Therefore,

$$T(n) = \Theta(n^2) \quad T(n) = \Theta(n^2) \quad T(n) = \Theta(n^2)$$

Recursion Tree Interpretation

Level 0

$$n \log n \quad n \log n \quad n \log n$$

Level 1

There are **4** subproblems.

Each size

$$n^2 \frac{n}{2} \frac{n}{2}$$

Cost

$$4(n^2 \log n) = 4 \left(\frac{n}{2} \log \frac{n}{2} \right) = 4(2n \log 2n)$$

Level 2

There are

$$4^2 = 16 \quad 4^2 = 16 \quad 4^2 = 16$$

subproblems.

The recursive work dominates the merging cost.

Hence,

Overall complexity becomes

$$\Theta(n^2) \quad \Theta(n^2) \quad \Theta(n^2)$$

Space Complexity

Recursion depth

$\log_2 n \log_2 n$

Each recursive level stores only constant extra information.

Therefore,

Space Complexity = $O(\log n)$

Performance Analysis

Inventory Size	Performance
Small inventory	Fast execution
Medium inventory	Moderate execution time
Large inventory	Time increases quadratically
Very large inventory	Computationally expensive

Although the algorithm divides the data efficiently, the four recursive calls increase the total execution time.

Asymptotic Analysis

Best Case

$\Omega(n^2)$

Average Case

$\Theta(n^2)$

Worst Case

$O(n^2)$

Advantages

- Suitable for divide-and-conquer processing.
- Can process inventory in parallel.
- Efficient recursive structure.
- Logarithmic recursion depth.

Disadvantages

- Four recursive calls increase execution time.
- Higher computational cost for massive datasets.
- Recursive overhead is present.

Final Result

Parameters

$a=4$ $a=4$ $a=4$ $b=2$ $b=2$ $b=2$ $f(n)=n \log n$ $f(n)=n \log n$ $f(n)=n \log n$

Master Theorem Case

Case 1 $\boxed{\text{Case 1}}$ Case 1

Time Complexity

$\Theta(n^2)$ $\boxed{\Theta(n^2)}$ $\Theta(n^2)$

Space Complexity

$O(\log n)$ $\boxed{O(\log n)}$ $O(\log n)$

Conclusion

The recurrence

$T(n)=4T(n/2)+n \log n$ $T(n)=4T(n/2)+n \log n$ $T(n)=4T(n/2)+n \log n$

fits the Master Theorem with $a=4$ $a=4$ $a=4$, $b=2$ $b=2$ $b=2$, and $f(n)=n \log n$ $f(n)=n \log n$ $f(n)=n \log n$. Since $n \log n$ grows more slowly than n^2 , **Case 1** applies. Therefore, the algorithm has a **time complexity of $\Theta(n^2)$** and a **space complexity of $O(\log n)$** due to the recursion depth.

